PCP OS

PCP OS Specification



The requirements, each numbered, each marked as built, partly built, or design.

SPECIFICATION

ENTERPRISE EDITION

Kirtus Justus

Creator and owner

Parrot Council Protocol (PCP) 1.0 · PCP OS, build 19 · 12 November 2026

Early development. Untested and not peer reviewed. Started only in an emulator. See EVIDENCE_STATUS.md.

Evidence before agreement.

PCP OS Specification

Product: Parrot Council Protocol (PCP), Version 1.0. **Component:** PCP OS, build 19. **Method carried:** Adversarial Claim Review (ACR) 1.0. **Creator:** Kirtus Justus. **Date:** 12 November 2026. **Status of this document:** a design specification for a system in early development. Most of it describes what is to be built. Each requirement says which.

0. How to read this

Each requirement is one numbered sentence. Most hold one requirement and one key word; where a sentence holds two joined by "and", the data form lists both key words. The key words MUST, MUST NOT, SHOULD and MAY are used as BCP 14 defines them (RFC 2119, RFC 8174), and bind only in capitals. Each requirement carries a status:

Status	Meaning
BUILT	Implemented, and checked by a test or self-test in this build. The check is named
PARTIAL	Part is implemented and checked; the sentence says which part is not
DESIGN	Not implemented. A statement of intent, and nothing may be claimed from it

A requirement marked DESIGN is not a property of PCP OS today. `PCP_OS_Requirements.json` is this section list as data.

PCP OS has not been peer reviewed, independently reviewed or evaluated in use. It has been started only in an emulator. Nothing in it is certified, and it is not fit for use where a failure can harm people.

1. Two paths

The central design rule. A computer exists to do the person's work. ACR is a method for examining claims. If the method's rules were checked on every operation, the processor would spend its time arguing about work instead of doing it. So the system has two paths that do not touch.

The **work path** is everything a program does: computing, reading, writing, drawing, talking to devices. No rule of ACR is evaluated on it.

The **record path** is the act of offering an event to a register: "this review was commissioned", "this check ran", "this finding was reached". A guard runs here, once per event, and nowhere else.

P-1. The kernel MUST NOT evaluate an ACR guard while handling a system call, an interrupt, a memory fault, a context switch or the completion of an input or output operation. *PARTIAL: system calls, the timer interrupt, faults from user mode and the switch between processes exist and evaluate no guard (`user.rs` calls none; the kernel records processes in its register after they have all ended). No input or output completion path exists.*

P-2. An ACR guard MUST be evaluated only when an event is offered to a register. *BUILT: the guards are reachable only through `Record::submit`, `Record::open` and `Record::extend` in the core.*

P-3. For a writer that has already checked a register, the cost of offering one more event MUST NOT grow with the number of events already in it, beyond reading the file. *BUILT: measured in this build at 0.28 ms per stored event at 100, 500 and 2,000 events, each event written durably; before this build it was 1.25 ms, 5.3 ms and 21 ms. Test: continuing a checked record equals checking the whole, at every split.*

P-4. While ACR is not in use, the system MUST do no ACR work beyond establishing that it is not in use. *BUILT for the host program: a hook with no register present reads nothing and writes nothing. DESIGN for the kernel.*

P-5. The kernel MUST NOT schedule, delay or reprioritise a task because of the state of a review. *PARTIAL: the scheduler takes the next process in turn and reads nothing else; the kernel holds no review, so the rule has not been tested against one.*

P-6. A halt of ACR use MUST be accepted in every state in which anything is in use. *BUILT: state-machine test over every state; host tests for a mismatched label, a register holding a refused event, and 64 open reviews.*

P-7. The system MUST start and be usable on a machine with no graphics processor, no network and no accelerator. *DESIGN: today it has no use for any of them.*

P-8. The system MUST NOT open a network connection that the person directing it did not cause. *PARTIAL: the LINK state machine starts OFFLINE and the guards refuse a retrieved artifact without its source; there is no network stack to enforce it on.*

2. The kernel

A new kernel for x86_64, written in Rust, with one file of assembly for entry and for interrupt stubs.

K-1. The kernel MUST contain no code from another kernel and MUST NOT require another operating system to start. *BUILT: its only dependency is the PCP OS core; it is started directly by a Multiboot loader.*

K-2. Before enabling a processor feature, the kernel MUST ask the processor whether it offers that feature, and MUST report and stop where a required one is absent. *BUILT: started as two processors without long mode; each printed the reason and halted.*

K-3. No memory MUST be both writable and executable. *BUILT: the page tables are read back and the count is zero; a write to code, a write to read-only data and execution of data are each attempted and refused by the processor.*

K-4. The kernel MUST map its own image and nothing else. *BUILT: 2,209 pages from 1 MiB to the end of the image; reads of the display memory, the low mebibyte and memory above the image are attempted and refused. Before this build the whole first gibibyte was mapped writable.*

K-5. The interrupt table, the segment table and the task state segment MUST be read-only once set. *BUILT: a write to each is attempted and refused.*

K-6. A double fault, a non-maskable interrupt and a machine check MUST each be delivered on a stack of their own, and every stack MUST have an unmapped page below it. *BUILT: a read of each of the five guard pages is refused (the kernel stack, the three fault stacks and the stack for entries from user mode); the stack-exhaustion demonstration ends in a reported double fault.*

K-7. The kernel MUST turn on supervisor execution prevention, supervisor access prevention and user-mode instruction prevention wherever the processor offers them. *BUILT: read back from the control register on a processor with all three and on one with none.*

K-8. An exception the kernel did not prepare for MUST stop the kernel with a report, and MUST NOT be continued past. *BUILT: the fault demonstration.*

K-9. Arithmetic overflow in the kernel MUST stop the kernel. *BUILT: overflow checks are compiled into the release build.*

K-10. Code that the compiler cannot check for memory safety MUST be confined to one named module, and each use MUST state why it is sound. *BUILT:* `arch.rs` *only; every other file refuses it at compile time.*

K-11. The kernel MUST record its own start, verification and stop in a register, and verification MUST end in OPERATIONAL or in SAFE and nothing else. *BUILT: the kernel's register replays with no problem.*

K-12. The kernel MUST measure its running image and compare the measurement with a value it did not compute. *PARTIAL: it measures. There is nothing yet to compare with: no signed image and no measured start.*

K-13. No code supplied by a third party MUST run in kernel mode. *DESIGN.*

K-14. The kernel SHOULD be loaded at an address that differs between starts. *DESIGN: it is loaded at a fixed address.*

K-15. The kernel MUST have a source of randomness before it issues any identifier or key. *DESIGN.*

Present since build 17: an address space for each program, user mode, five system calls through one gate, a round-robin scheduler driven by the timer, and a budget after which a process is stopped. Each is checked by a self-test in which a program attempts the thing and the kernel reports what the processor or the kernel itself answered. The programs are thirteen short sequences of instructions built into the image.

Not present: a way to load a program from outside the image, a program format, more than one page each of code, data and stack, memory that is returned when a process ends, floating-point and vector state kept between processes, messages between processes, storage, a network stack, drivers beyond a serial port and a timer, a second processor, start from UEFI. It has not been started on hardware.

3. Authority and administration

A-1. Authority MUST be held as capabilities, and a capability derived from another MUST carry the same rights or fewer. *BUILT in the core: derivation, narrowing and revocation tests. PARTIAL in the kernel: each process is a holder, and the write call is answered only for a process that holds a capability for the console (self-test: a program started without it is refused). One object and one right; no capability passes between processes.*

A-2. There MUST be no identity that holds every right. *DESIGN.*

A-3. No combination of roles MUST give a model or a program the right to start ACR use, to administer, or to change the system. *BUILT: exhaustive over every kind of identity, set of roles and edition.*

A-4. Administration MUST be a role held by a person and exercised only while the system is in MAINTENANCE. *PARTIAL: the SYSTEM state machine has MAINTENANCE and its transitions, and entering it asks every task to stop. Nothing yet enforces it on a running system.*

A-5. Every administrative act MUST be an event in the register before it takes effect. *DESIGN.*

A-6. In the Commercial and Enterprise Editions, whoever administers an installation MUST NOT assess under it. *PARTIAL: stated in the edition profile; the host program refuses a two-person standard in the Personal Edition; separation of the two roles is not enforced.*

4. What of ACR can be changed, when, and by whom

The balance between the method and the machine is settled by one fact: everything is identified by a digest, and every record says which digests it was made under. Nothing therefore needs to be forbidden to the person who owns the machine. What must hold is that a change is never silent.

Thing	Who may change it	When	What the record shows
The native guards	The person, by replacing the kernel image	In MAINTENANCE	A different image measurement in every later record
The method release	Nobody in place. Another release is another identity	Between reviews	The release each register is pinned to, and whether the text at hand was verified against it
A released module	Nobody in place. A changed copy is another module	Between reviews	Its own digest
A module of the person's own	The person	Between reviews	Its identifier and digest
The workspace's modules	The person directing the work	Any time; applies to reviews commissioned afterwards	Who selected them and how the host knows
A standard	A person, acting as themselves	Any time	It narrows what the method allows and never widens it
An open review's method and modules	Nobody	Never while it is open	Fixed at commission

M-1. Every register MUST state the method release it is held to, by digest. *BUILT.*

M-2. A review MUST keep the method release and the modules it was commissioned under until it is closed. *BUILT for modules, which are fixed in the commission event. PARTIAL for the release, which is fixed per register.*

M-3. Adding or removing a module MUST require no change to any program. *BUILT in the host program, the plugins and the machine form: a module is found by the header of its own Specification, and a test adds one the program has never heard of. The method packet's own Python tools still carry a list.*

M-4. A commission MUST name each selected module by identifier, version and the digest of its Specification, and each MUST be a module found in the method text at hand. *BUILT: tests for a wrong digest, a wrong version, an unknown module, Core listed, and a module listed twice.*

M-5. A module written for another Core than the one a program implements MUST NOT be selected. *BUILT.*

M-6. A participant MUST NOT select a module; selection is the act of the person directing the work or of a policy they set. *BUILT: recorded by the host from a typed command or a terminal; refused from a session.*

M-7. A licensee MAY modify a module or add their own. *PARTIAL: adding is a supported path in the host program (`pcpos module add`): the module stands beside the register, the addition is recorded with the digest of its Specification, and a review selects it by that digest. Modifying a released module is not a supported path: a changed file is found, its real digest is used, and the register records that the method text at hand differs from the release.*

M-8. A module that is not a released module MUST NOT carry the identifier of one. *PARTIAL: the host program accepts an added module only under an identifier beginning `local.`, refuses one that takes the name of a module of the method, and serves no text for an identifier that two folders claim. A folder placed by hand inside the method text itself and named `acr.<new name>` is still listed as a module; the register's method check then reports the text as differing from the release.*

M-9. The person who owns the machine MAY replace any program outside the kernel image and MAY rebuild the image. *DESIGN.*

5. Languages and trust

The weight a result carries comes from what executed the check and what the record holds about it. It does not come from the name of a language, and no program asks which language something is written in. The difference between languages is expressed once, structurally, as where code of each kind is allowed to run and what it is allowed to hold.

Tier	What runs there	Rule
T0	The kernel image	Rust, no standard library, uncheckable code only in the one named module
T1	System services: registrar, store, the host program	Rust, uncheckable code refused at compile time
T2	Contained native code: C, C++, assembly, vendor drivers, existing programs	Runs only inside a compartment, holding only the capabilities it was handed
T3	Interpreted and managed code: Python and others	The interpreter is a T2 program. A script is data it was given

L-1. A record MUST state what produced each observation it holds: which participant, and whether the host observed it or a participant reported it. *BUILT: actor, authority,* `read_by`, `observed` *and the task worker are recorded. Since build 18 a check says how the record came by it:* `HOST_OBSERVED_TOOL` *where the host saw the shell command run and passed on its exit status and output,* `REPORTED_BY_PARTICIPANT` *where a participant gave its own account, which is all that builds before 18 could record. The host's observation is of the command and its output as the host reported them; it is not an observation that the command tests anything. Since build 19 the observation must have been made after the open review was commissioned, a finding that rests only on reported checks is marked* `execution: UNKNOWN`*, and material that reaches the record while a review is open is recorded by how it came (words the host did not see the person type, and a file written since commissioning, as PRODUCED). The observations are kept in an ordinary file that a shell command can write: none of this is authentication.*

L-2. System services MUST NOT hold a capability to an interpreter or a compiler. *PARTIAL: the host program depends on no interpreter and runs none, clause lookup included. Capabilities between services do not exist yet.*

This is why the system does not use Python for its own checks: not by a rule against Python, but because nothing in T0 or T1 can reach it.

L-3. Where two implementations of the core disagree, the Rust implementation MUST govern. *BUILT: the Python implementation is run against 17,113 cases published by the Rust one.*

L-5. A finding MUST NOT be recorded as supported or as refuted where all that it cites is text the review itself produced. *BUILT: both implementations refuse it (Core 4.3, 5.2), for an artifact recorded as produced after the review was commissioned and for what a task of the review returned. What the guards cannot do is tell a claim that a procedure settles from one that reading settles: the participant declares the kind of claim, and the status shows for each finding whether a check was run and who saw it.*

L-4. Code in C or C++ MUST run in T2. *DESIGN.*

C and C++ are used where Rust does not fit: existing device code, vendor drivers, programs that already exist. Their memory errors are then contained by the compartment and not by the hope that there are none.

L-5. The core MUST have one stable binary interface, in the C calling convention, and every binding for another language MUST be a thin layer over it that adds no rule. *DESIGN: today the core is reached as a Rust library, through the tool server and at a terminal. The Python implementation is a second implementation, not a binding, and is to become one.*

6. Interfaces

I-1. Every way of reaching a register MUST go through the same core and the same guards. *BUILT: the terminal commands, the tool server and the hooks are one program over one core.*

I-2. A caller MUST NOT be able to name the actor of an event. *BUILT.*

I-3. The interface for models MUST be the Model Context Protocol. *BUILT: revisions 2024-11-05 to 2025-11-25; tools only.*

I-4. Material retrieved from outside MUST be recorded with where and when it was retrieved. *BUILT in the guards.*

There is deliberately no separate application for connecting PCP OS to other programs' interfaces. A second gateway would be a second place where the rule about who is acting has to be right. Integration is done by speaking to the registrar through one of its interfaces: the tool server for models, the binary interface of L-5 for programs.

7. Extensions

PCP OS is meant to be extended and changed by the person who holds it. The question is how to allow that without the two failures other systems show: extensions that run with the system's own authority, and extensions that nobody can account for.

X-1. An extension MUST be a package with a manifest stating its identity, its digest, its tier and the capabilities it asks for. *DESIGN. The two PCP plugins for Claude are packages of this shape for another host.*

X-2. An extension MUST NOT run in kernel mode. *DESIGN.*

X-3. An extension MUST receive only the capabilities the person granted it, and installing, granting and revoking MUST each be recorded. *DESIGN.*

X-4. Revoking a capability MUST take back everything derived from it, in every holder. *BUILT in the core.*

What other systems teach, stated from memory and not checked for this document: kernels that load third-party modules into kernel mode inherit those modules' faults as their own; signing a driver establishes who shipped it, not that it is safe; systems that keep drivers and services outside the kernel and hand them capabilities contain such faults; an extension model with no containment turns every extension into a supply-chain risk.

8. Devices, display, time and first start

D-1. The system MUST use the display at its native resolution as the firmware reports it, and SHOULD choose the default scale from the display's physical density. *DESIGN: the kernel has no display code.*

D-2. Default backgrounds MUST be drawn at the display's own resolution from a resolution-independent description, and MUST use the design tokens of the edition and the workspace. *PARTIAL: the backgrounds exist as vector files generated from the tokens. Nothing displays them.*

D-3. Every record MUST carry time in UTC. *BUILT.*

D-4. The time zone MUST be a setting of each person, used for display only, and setting it MUST NOT need a network. *DESIGN.*

D-5. First start MUST create the person who directs the system before anything else is allowed, and MUST NOT require an account with any service. *DESIGN.*

Intended order of first start: verify; name the person; read the edition; ask language, keyboard and time zone once, proposing what the firmware states; optionally set a workspace; become OPERATIONAL. Each step is an event.

D-6. A graphics processor MUST be a device like any other: used by whoever holds a capability to it, absent without consequence. *DESIGN.*

9. Security

What is defended, and what is not.

Threat	Today
A model or program writing events it has no right to, through the interfaces	Refused: actor fixed per session; person's and system's events not accepted from a session
A model getting its own reading of a prompt recorded as the person's act	Refused since build 13: only a typed command is the person's
Reaching a file outside the served directory	Refused by name and by the opened object's real location (Linux); a race can still create an empty file outside
Rewriting the register file directly	Not prevented on another operating system. Detected only by someone who kept the head digest
Running the host program with made-up input	Not prevented on another operating system
Corrupting kernel memory through a kernel bug	Limited by K-3 to K-7; not by address randomization or stack canaries
A changed kernel image	Measured, not compared (K-12)
Speculative-execution attacks	No mitigation beyond the processor's own
Devices that write memory directly	No protection
Physical access	No protection

Mitigation	Status
Writable or executable, never both	BUILT
Least mapping; guard pages; separate stacks for three exceptions	BUILT
Read-only processor tables	BUILT
Supervisor execution and access prevention	BUILT where offered; user memory now exists, and the kernel reads a program's bytes through its own mapping and never through a user address

User mode: separate address spaces, one system-call gate, no device ports, preemption, a budget	BUILT, as self-tests in the emulator
Privilege separation by capability	BUILT in the core; PARTIAL in the kernel (one object, one right)
Address randomization; stack canaries; randomness	DESIGN
Signed image; measured start	DESIGN
Containment of native code (T2)	DESIGN

10. Order of work

One thing at a time, each finished and checked before the next: address spaces and user mode (done in build 17); a program format and a loader, so that a program need not be part of the image; capability-checked message passing; the registrar as the first service, with storage; start from UEFI with a framebuffer; a measured start; the binary interface of L-5; compartments for T2.